Experiment 10: Machine Learning Lifecycle and Deployment using MLflow

Experiment No.	10	Mapped CO	CO5
Subject	Agile Software Development and DevOps Lab	Course Code	2175119

Aim

To track machine-learning experiments, version models and deploy a model using MLflow.

Tools / Software

Python 3.10 or above; MLflow; scikit-learn; sample dataset; terminal / command prompt; web browser.

Theory

MLOps applies DevOps principles to the machine-learning lifecycle, including data preparation, model training, evaluation, deployment and monitoring. A major challenge in machine learning is reproducibility because the final result depends on the dataset, source code, parameters and execution environment.

MLflow supports reproducible machine-learning workflows through four major components: Tracking, Projects, Models and Model Registry. MLflow Tracking records parameters, metrics and artifacts for every run. The Model Registry supports model versioning and stage transitions such as Staging and Production. The selected model can be served behind a REST endpoint for prediction requests.

Procedure

1. Install MLflow and required Python libraries such as scikit-learn.
2. Start the MLflow tracking UI using the mlflow ui command.
3. Create a training script and start an MLflow run in the script.
4. Train a machine-learning model and log hyperparameters, metrics and model artifacts.
5. Compare runs in the MLflow UI and identify the best run based on evaluation metrics.
6. Register the best model in the MLflow Model Registry.
7. Serve the registered or selected model using mlflow models serve and send a prediction request.
8. Paste screenshots or copied outputs in the observation boxes provided below.

Sample Code / Commands

train.py - MLflow tracking sample

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

with mlflow.start_run():
    model = RandomForestClassifier(n_estimators=100)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
```

```
    mlflow.log_param('n_estimators', 100)
    mlflow.log_metric('accuracy', acc)
    mlflow.sklearn.log_model(model, 'model')
```

MLflow UI and model serving commands

```
mlflow ui
mlflow models serve -m runs:/<run_id>/model -p 1234 --no-conda

# Example prediction request after serving
curl -X POST http://127.0.0.1:1234/invocations \
  -H "Content-Type: application/json" \
  -d '{"dataframe_split":{"columns":["f1","f2"],"data":[[1,2]]}}'
```

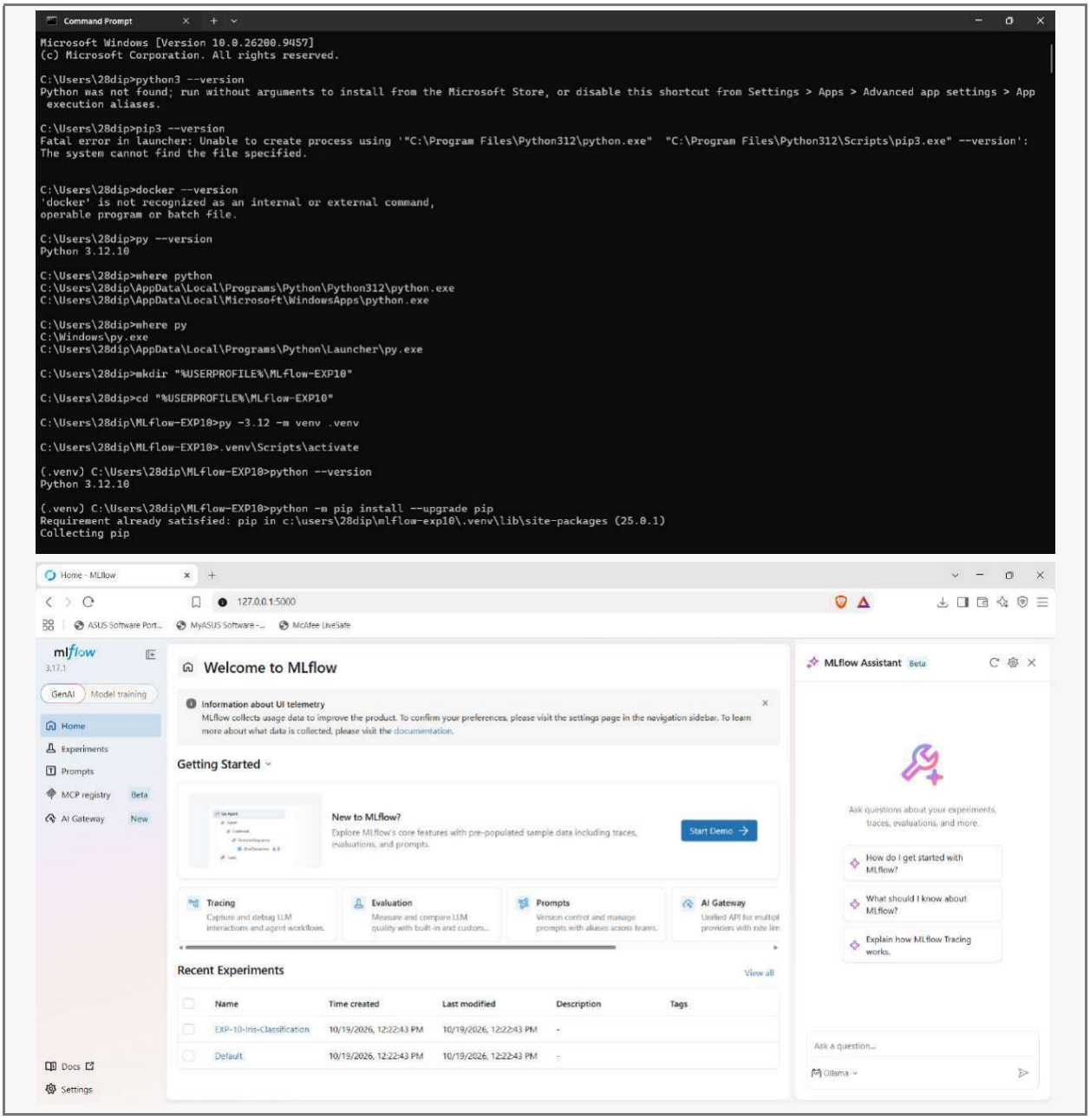
Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing MLflow installation, experiment tracking, model registration and model serving. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Vighnesh Pingale	24107108	B2	19-10-2026

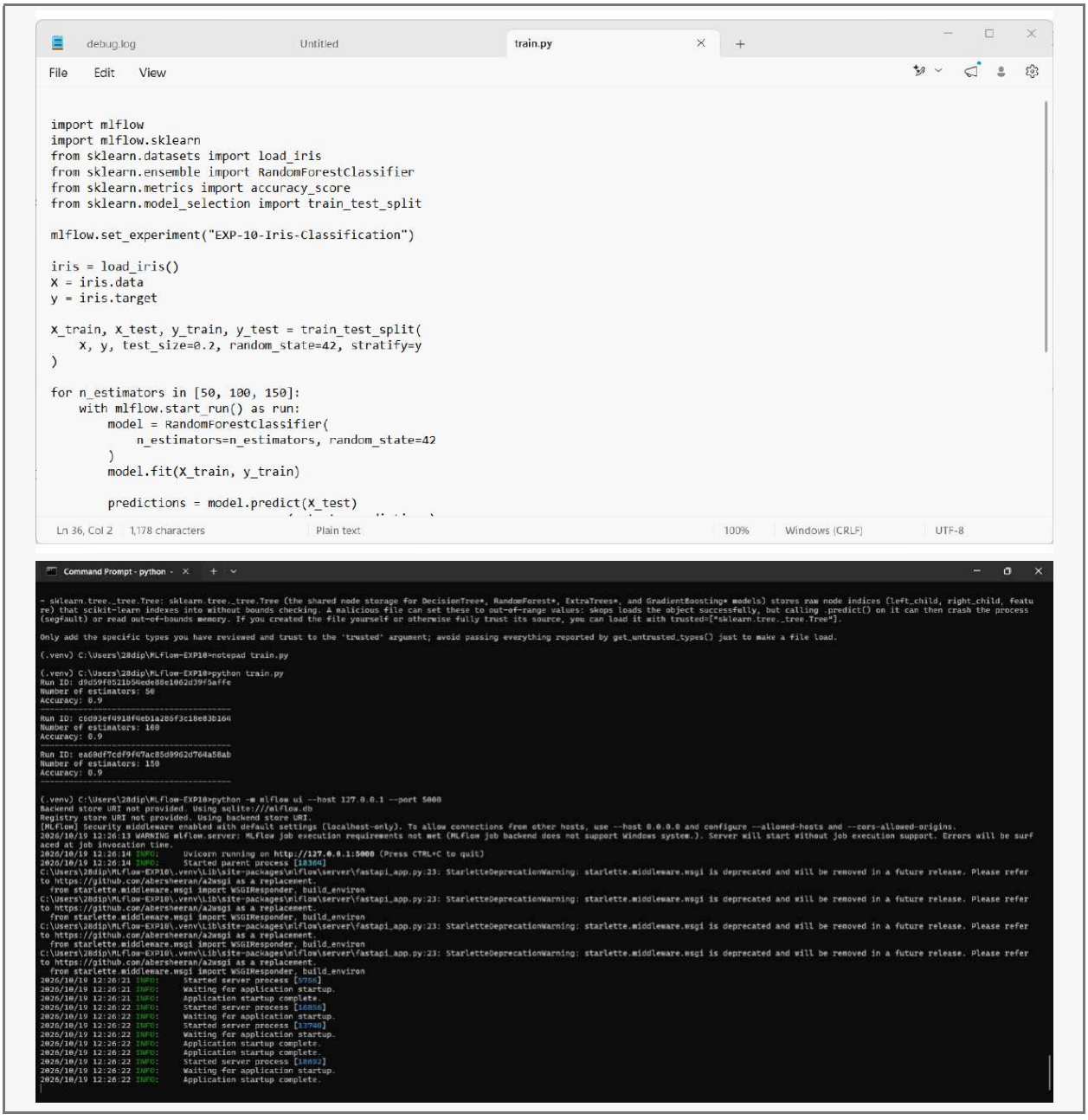
1. MLflow Installation and Tracking UI Output

Paste screenshot/output showing successful MLflow installation and the MLflow tracking UI running in the browser.



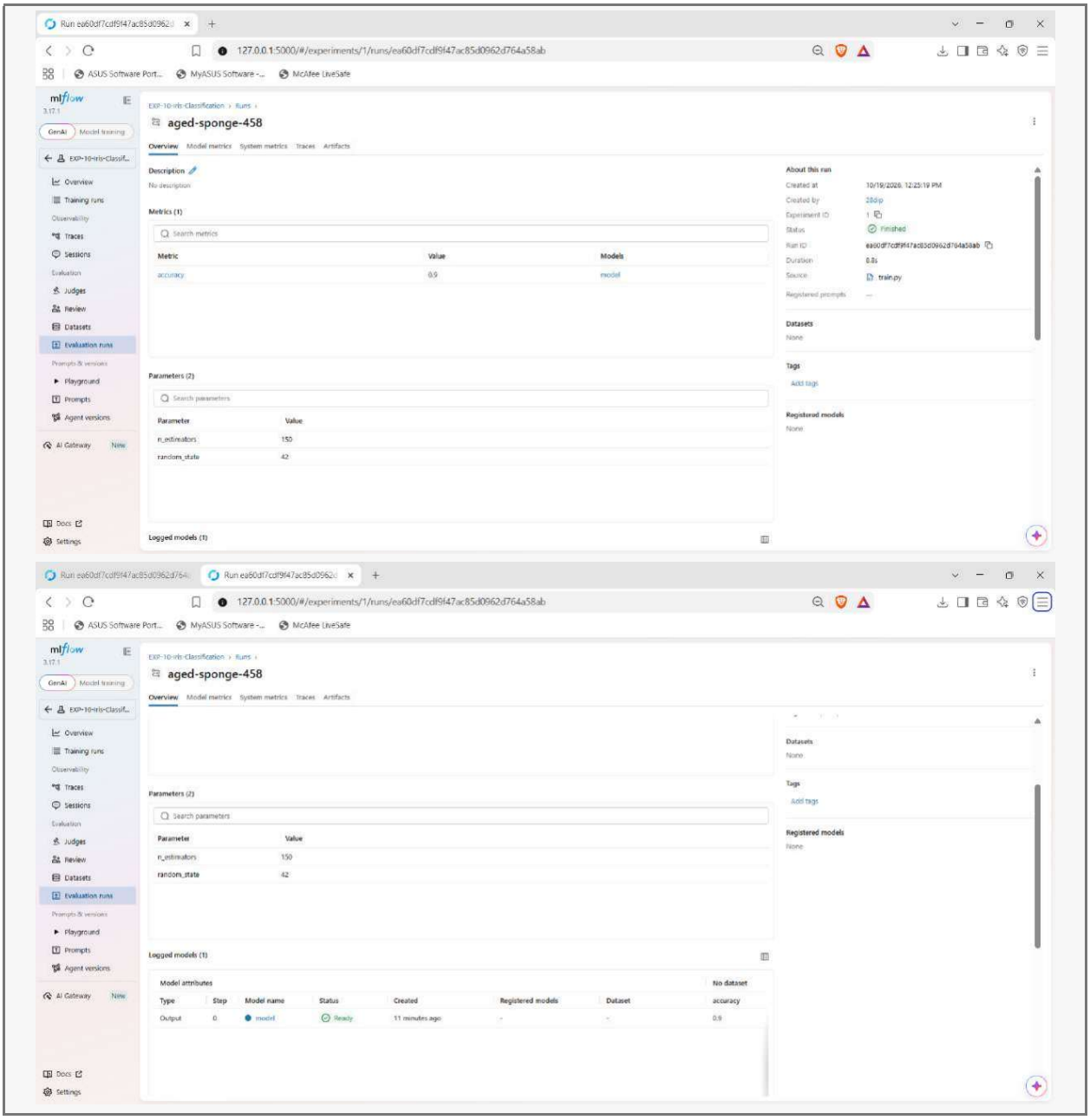
2. Training Script Execution Output

Paste screenshot/output showing execution of train.py and successful completion of a model training run.



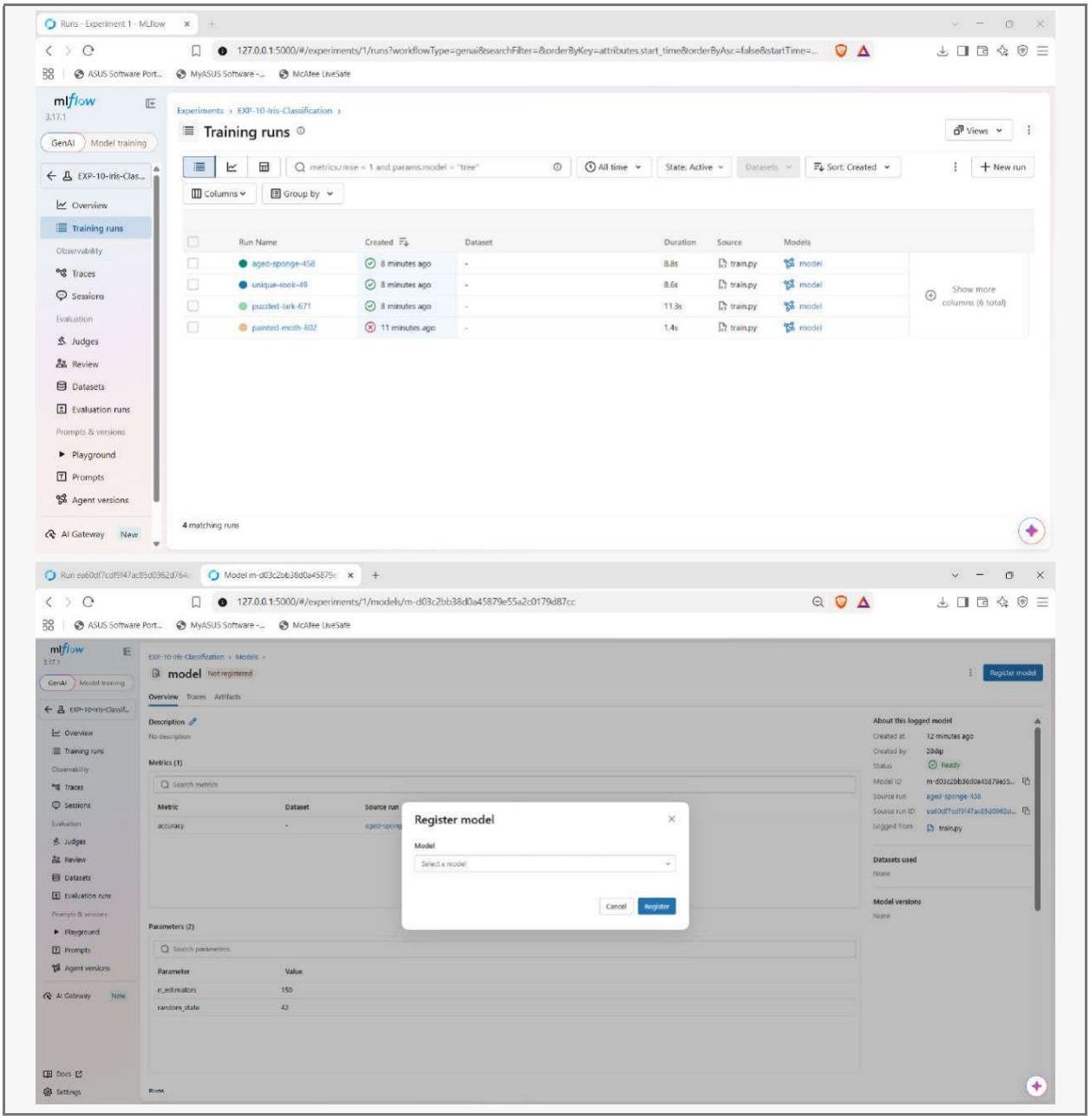
3. Parameters, Metrics and Artifacts Output

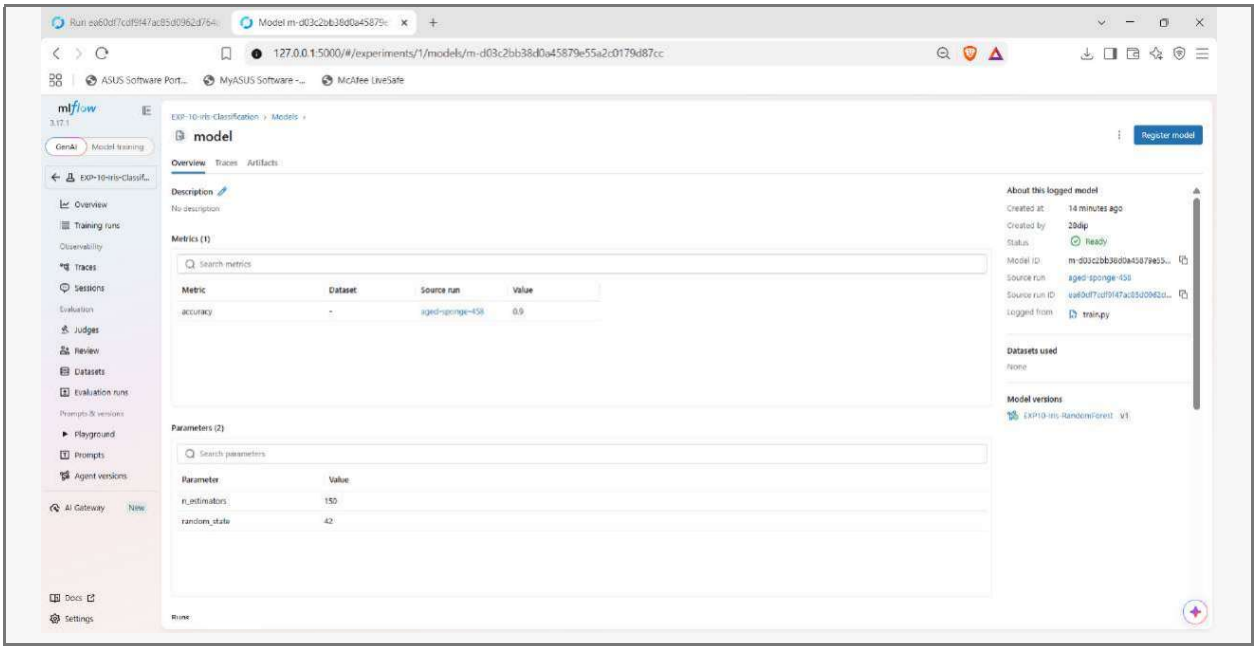
Paste screenshot/output showing logged parameters, accuracy or other metrics, and model artifact in the MLflow UI.



4. Run Comparison and Model Registry Output

Paste screenshot/output showing comparison of runs and registration of the best model in the Model Registry.




127.0.0.1:5000/#/experiments/1/models/m-d03c2bb38d0a45879e55a2c0179d87cc
mlflow
model
Register model
Overview
Traces
Artifacts
Description
No description
Metrics (1)
Metric
Dataset
Source run
Value
accuracy
aged-sponge-458
0.9
Parameters (2)
Parameter
Value
n_estimators
150
random_state
42
About this logged model
Created at
14 minutes ago
Created by
20dip
Status
Ready
Source run
aged-sponge-458
Logged from
train.py
Datasets used
None
Model versions

5. Model Serving and Prediction Output

Paste screenshot/output showing mlflow models serve command and prediction response from the served REST endpoint.



MLflow Tracking UI URL:

http://127.0.0.1:5000

Model Serving Endpoint URL:

http://127.0.0.1:1234/invocations

GitHub Repository / Training Script Link:

Not uploaded to GitHub - training script kept locally (C:\Users\28dip\MLflow-EXP10\train.py).

Final Observation Statement

MLflow was installed in a Python virtual environment and the tracking UI was started on port 5000. train.py trained a RandomForestClassifier on the Iris dataset with 50, 100 and 150 estimators, logging the parameters, accuracy (0.9) and model artifact for each run. The runs were compared in the MLflow UI and the model from run aged-sponge-458 was registered as EXP10-Iris-RandomForest (version 1). The registered model was served with mlflow models serve on port 1234 and a curl prediction request returned a valid response.

Expected Output

Each training run appears in the MLflow UI with logged parameters, metrics and model artifact. Runs are comparable, the best model is registered, and the served endpoint returns predictions for posted input.

Conclusion

The machine-learning lifecycle was managed using MLflow by logging and comparing experiments, versioning a model in the registry and serving the model, demonstrating reproducible MLOps practice.

Viva / Review Questions

1. What problem does experiment tracking solve?
2. Name the four components of MLflow.
3. What is the purpose of the Model Registry?
4. How does MLOps differ from traditional DevOps?

Marks: ____ / 10	Date: _____	Signature of Faculty: _____